

Programmazione 1

Corso c

04/12/2023

> Ricorsione e correttezza (3)

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica

Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Induzione per la correttezza della ricorsione
- Ricerca dicotomica ricorsiva e correttezza
- Merge Sort e correttezza

Come si progetta un metodo ricorsivo

- Vogliamo calcolare una funzione matematica f definita su numeri naturali usando un metodo F ricorsivo.
- Il metodo F deve definire almeno due casi:
 - Caso base della ricorsione: $n == 0$, in questo caso si restituisce il valore di $f(0)$.
 - Passo ricorsivo: $n > 0$, in questo caso la tecnica permette l'uso dell'invocazione $F(n-1)$ per calcolare il valore di $F(n)$.

Si può assumere che questa invocazione restituisca correttamente il valore $f(n-1)$ quindi, se abbiamo scritto con attenzione il codice per il passo ricorsivo, **mediante induzione si dimostra allora che $F(n)$ calcola correttamente il valore di $f(n)$.**

Ricorsione e Induzione

Ricorsione: F

- Base della ricorsione: se $n=0$ si restituisce il valore $f(0)$
- Passo ricorsivo: se $n>0$, si restituisce un valore rappresentato da una espressione che, solitamente, conterrà la sottoespressione $F(n-1)$

L'espressione per il valore di $F(n)$ è suggerita da una definizione ricorsiva per la funzione f .

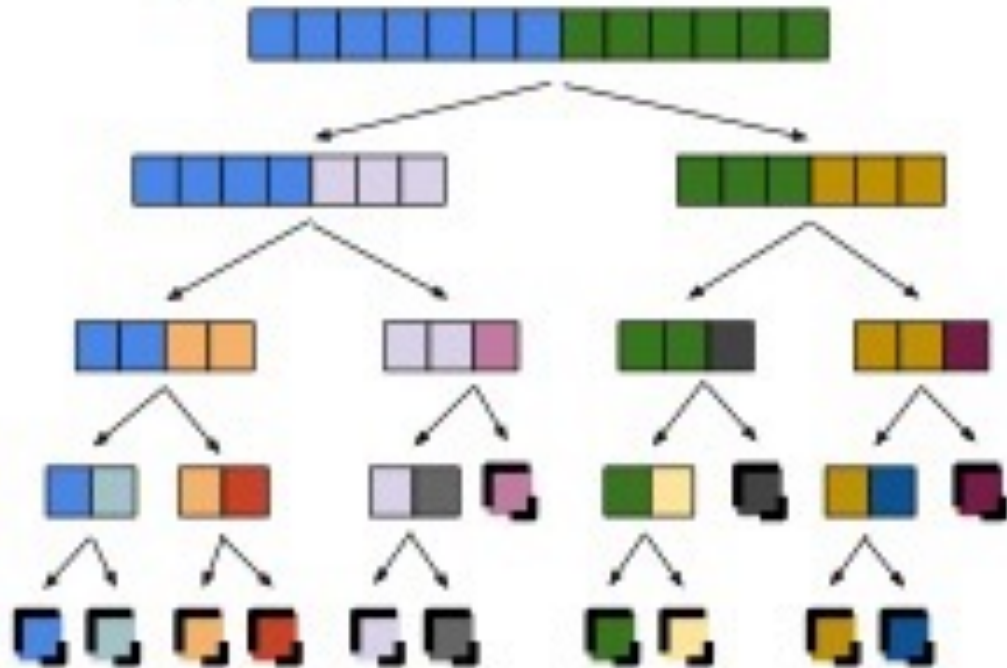
Induzione:

- $F(0)=f(0)$
- Passo induttivo: assumendo che $F(n-1)$ calcoli correttamente $f(n-1)$ (ipotesi induttiva), si dimostra che $F(n)$ calcola correttamente $f(n)$.

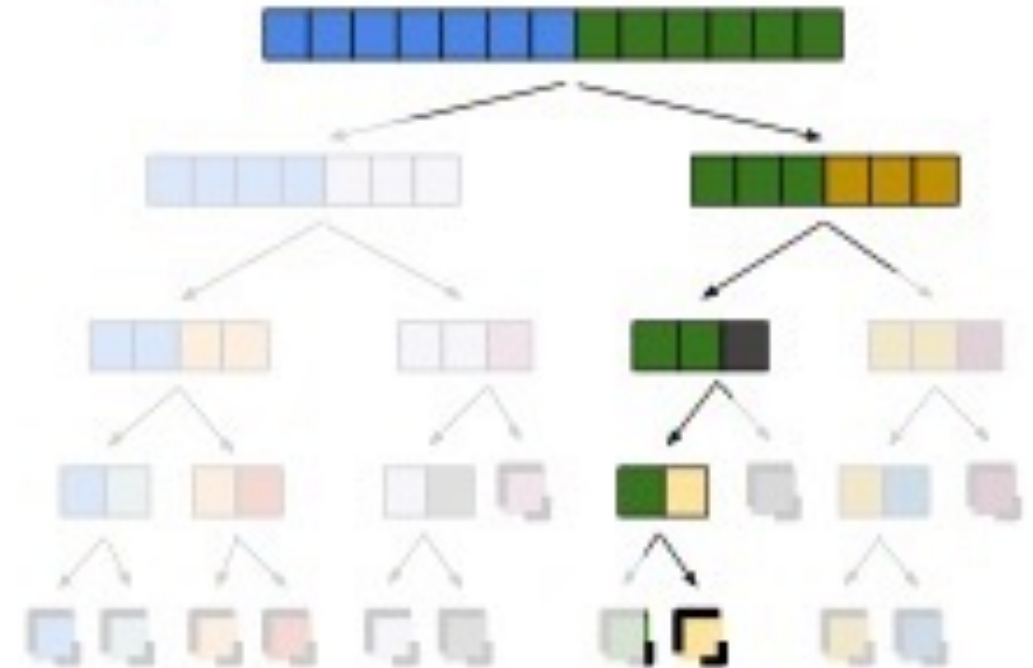
La dimostrazione del passo induttivo sfrutta la definizione ricorsiva della funzione F

Ricerca dicotomica

Esempio ricorsione dicotomica:



Esempio ricerca dicotomica:

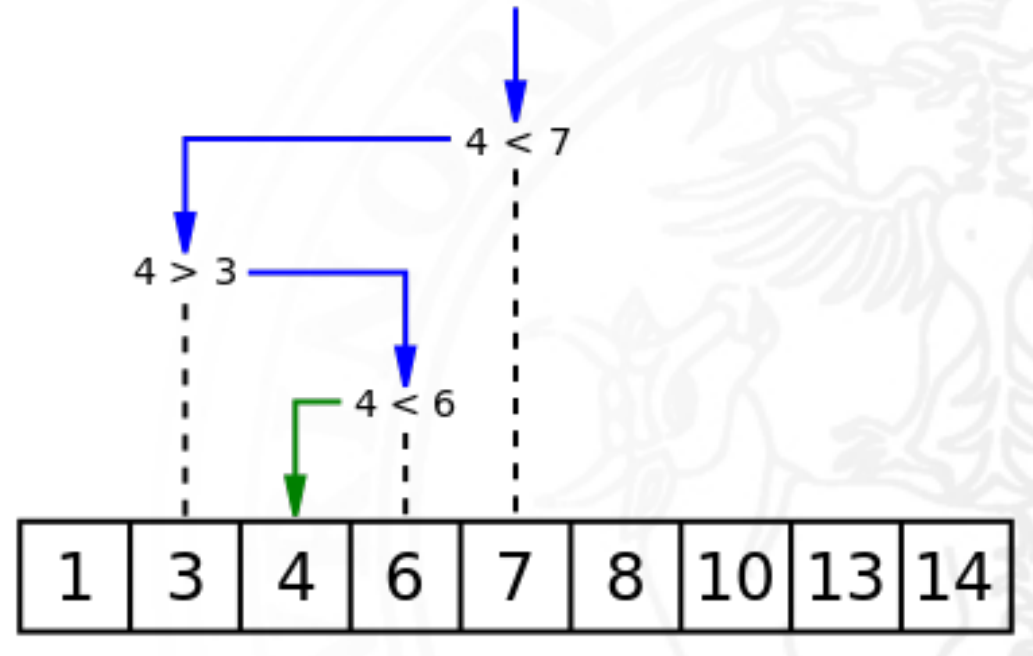


Ricerca dicotomica

- L'algoritmo di ricerca binaria cerca un elemento in una sequenza ordinata in maniera crescente (o decrescente)
- L'algoritmo è simile al metodo usato trovare una parola sul dizionario: l'idea è quella di iniziare la ricerca dall'elemento centrale.
- Si confronta questo elemento con quello cercato:
 - se corrisponde, la ricerca termina;
 - se è superiore, la ricerca viene ripetuta sugli elementi precedenti (ovvero sulla prima metà del dizionario);
 - se invece è inferiore, la ricerca viene ripetuta sugli elementi successivi (ovvero sulla seconda metà del dizionario)
- La ricerca binaria non usa mai più di $\log_2 N$ confronti.

Ricerca dicotomica

- Ricerca dell'elemento 4:



Ricerca dicotomica

```
int ricercaBinariaDic(const int len, const int a[],const int valore,const int
                        left,const int right) {
    if(right==left+1) {
        if(a[left]==valore) {
            return left;
        }
        else {
            return -1;
        }
    }
    else {
        int middle = (left+right)/2;
        if(valore < a[middle]) {
            return ricercaBinariaDic(len,a,valore,left,middle);
        }
        else {
            return ricercaBinariaDic(len,a,valore,middle,right);
        }
    }
}
```


Correttezza: caso base

Per ipotesi, l'array è ordinato (precondizione).

- Se $n==1$, allora ho $left+1==right$. Quindi il programma termina (non ci sono loop o chiamate) e ritorna `left` se `valore==a[left]`, oppure `-1` se `valore!=a[left]`

Correttezza: caso induttivo (*strong induction*)

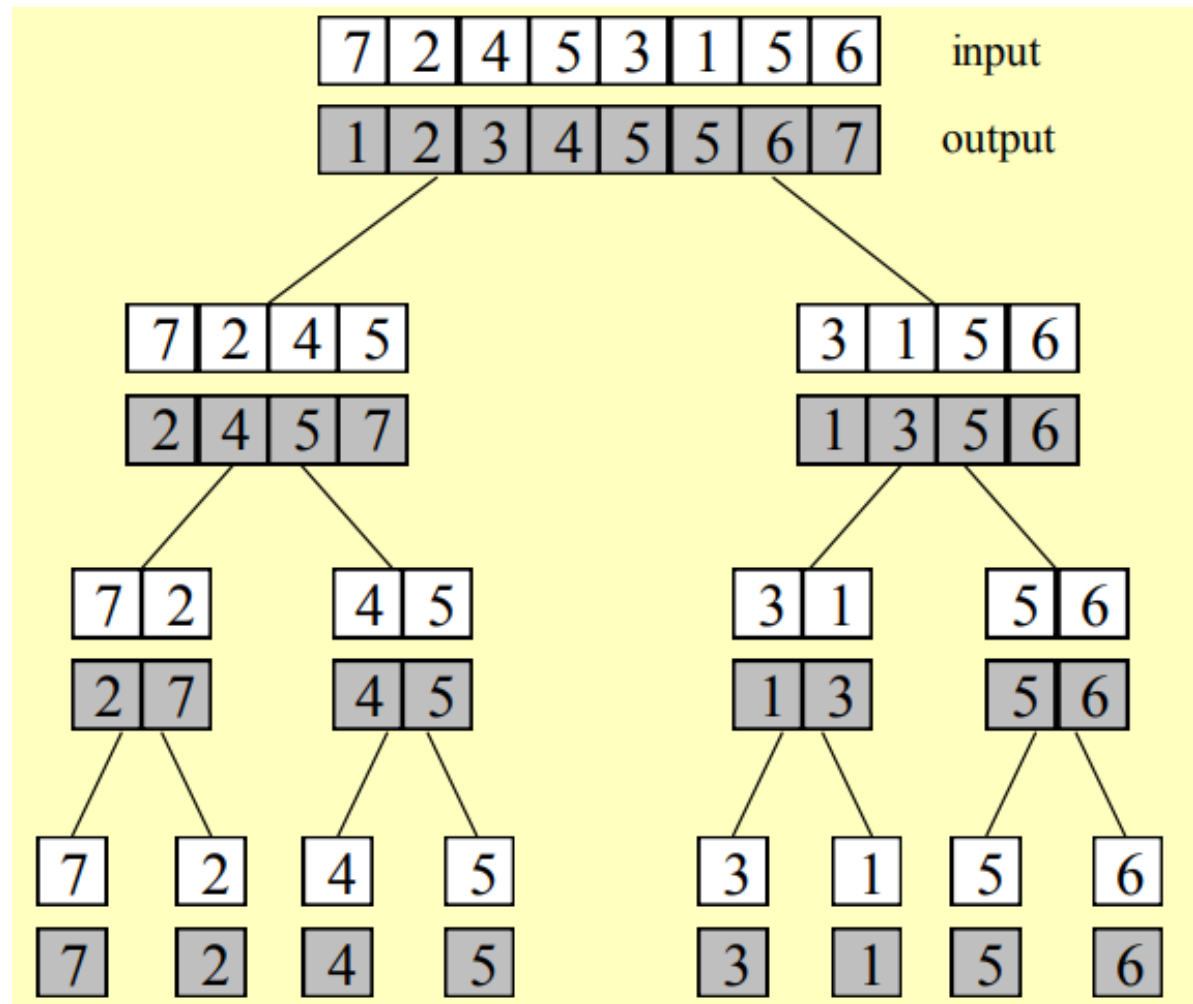
Se $n > 1$ allora $left + 1 \neq right$, il primo `if` fallisce. Nel secondo `if` ho 2 possibilità:

1. `valore < a[middle]`: poiché l'array è ordinato, sappiamo che `valore` potrebbe essere solo in `a[left, middle)`. Ma, per ipotesi induttiva, la chiamata ricorsiva su `[left, middle)` dà il risultato corretto.
2. `valore >= a[middle]`: similmente, poiché l'array è ordinato, sappiamo che `valore` potrebbe essere solo in `a[middle, right)`. Ma, per ipotesi induttiva, la chiamata ricorsiva su `[middle, right)` dà il risultato corretto.

Merge Sort

- L'algoritmo fu inventato da John von Neumann nel 1945
 - https://it.wikipedia.org/wiki/Merge_sort
- Usa la tecnica del divide et impera:
 1. **Divide**: dividi l'array a metà;
 2. Risolvi i due sottoproblemi ricorsivamente;
 3. **Impera**: fondi le due sottosequenze ordinate.
- Complessità in tempo $O(n \log n)$... lo dimostrerete all'esame di algoritmi!

Merge Sort



Merge Sort: le chiamate ricorsiva

```
void mergeSort(const int lA, int a[], const int left, const int right)
{
    if (right != left + 1)
    {
        int middle = (left + right) / 2;
        mergeSort(lA, a, left, middle);
        mergeSort(lA, a, middle, right);
        merge(lA, a, left, middle, right);
    }
}
```

Merge Sort: il *merge*

Due (sotto)array ordinati A e B possono essere fusi rapidamente.

Tecnica iterativa:

1. estrai ripetutamente il minimo di A e B e copialo nell'array di output, finché A oppure B non diventa vuoto;
2. copia gli elementi dell'array non vuoto alla fine dell'array di output

Merge Sort: *il merge*

```
void merge(const int lA, int a[],const
int left, const int middle,const int
right)
{
    int i=left;
    int j=middle;
    int k=left;
    int b[lA];
    while(i<middle && j<right) {
        if(a[i] < a[j]){
            b[k] = a[i];
            i++; }
        else {
            b[k] = a[j];
            j++; }
        k++;
    }
```

```
while(i<middle)
{
    b[k] = a[i];
    i++;
    k++;
}
while(j<right)
{
    b[k] = a[j];
    j++;
    k++;
}

for(int i=left;i<right;i++)
    a[i]=b[i];
}
```



Correttezza: caso base

- Se $left+1==right$ allora c'è un solo elemento, la funzione non fa nulla poiché l'array risulta già ordinato.

Correttezza: caso induttivo

- Supponiamo che ci siano n elementi nell'array e che per $n-1$ si riesca ad ordinarli correttamente
- Quindi `mergeSort(lA, a, 0, middle)` e `mergeSort(lA, a, middle, right)` e ordinano correttamente gli intervalli `[0, middle)` e `[middle, right)`.
- La funzione iterativa `merge(lA, a, left, middle, right)` prende i due intervalli `[0, middle)` e `[middle, right)`, fondendoli in nuovo intervallo che ordina l'array iniziale, usando un array di buffer.
- **N.B.** si *dovrebbe* dimostrare quest'ultimo punto formalmente (es. weakest precondition), ma supponiamo di averlo fatto. *Merge ricorsivo?*

Merge Sort: il *merge* ricorsivo

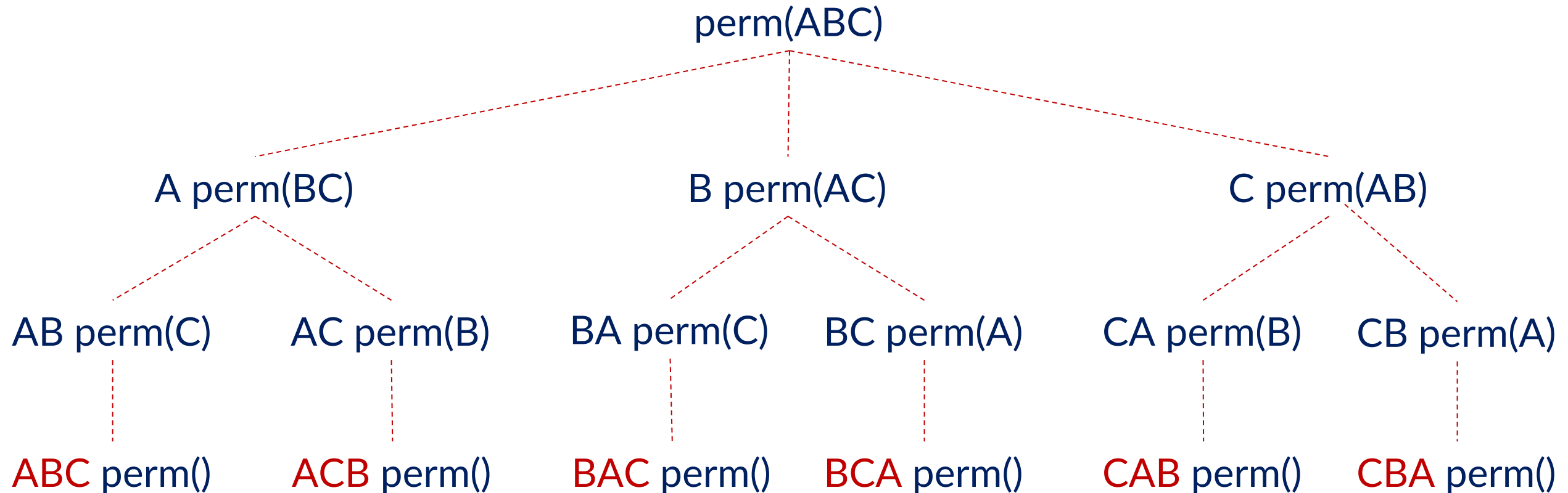
```
void mergeR(const int lA, int a[],const int left,
const int middle,const int right)
{
    if(left == middle) {}//sx vuoto, finito
    else if(middle==right){}//dx vuoto, finito
    else if(a[left] <= a[middle])
        { //primo sx <= primo dx
            mergeR(lA,a,left+1,middle,right);
        }
    else
        { //primo sx > primo dx
            int buffer = a[middle];
            shiftOneRight(lA,a,left,middle);
            a[left] = buffer;
            mergeR(lA,a,left+1,middle+1,right);
        }
}
```

```
void shiftOneRight(const int lA, int
a[],const int left, const int middle)
{
    if(left==middle){}
    else
        {
            a[middle]=a[middle-1];
            shiftOneRight(lA,a,left,middle-1);
        }
}
```

Esercizio: Anagrammi

- Scrivere una funzione ricorsiva/iterativa che stampi tutti gli anagrammi, cioè tutte le permutazioni, di una stringa.
- ABC->ABC ACB BAC BCA CAB CBA
- `length!` permutazioni
- Cerchiamo l'algoritmo ragionando su un caso facile ...

Permutazioni iterative-ricorsive



Funzioni ausiliarie per anagrammi

- Per eliminare un carattere (*inplace*)
- Copiare una stringa
- Concatenare un carattere alla fine di una stringa

Anagrammi

```
void anagrammiR(char* s, char* prefix) {
    if(*s=='\0') {
        printf("%s\n", prefix);
    }
    else {
        int i=0;
        while(*(s+i)!='\0') {
            char s2[DIM];
            scopy(s, s2);
            eliminaCarattere(s2, i);
            char prefix2[DIM];
            scopy(prefix, prefix2);
            concatenaStrCh(prefix2, *(s+i));
            anagrammiR(s2, prefix2);
            i++;
        }
    }
}
```

```
void eliminaCarattere(char* s, int i) {
    eliminaCarattereR(s, s, 0, i); }

void eliminaCarattereR(char* s, char* d, int p, int i) {
    if(*s=='\0') {
        *d=*s; }
    else if(p==i) {
        eliminaCarattereR(s+1, d, p+1, i); }
    else {
        *d=*s;
        eliminaCarattereR(s+1, d+1, p+1, i); }
}

void scopy(char* s1, char* s2) {
    *s2=*s1;
    if(*s1!='\0') {
        scopy(s1+1, s2+1); }
}

void concatenaStrCh(char *s, char c) {
    if(*s=='\0') {
        *(s+1)=*s;
        *s=c; }
    else {
        concatenaStrCh(s+1, c); }
}
```

Esercizi ricorsivi

- Su interi:
 - Funzione di un numero naturale n che restituisce la somma di tutti i divisori di n , escluso n .
- Su array:
 - Funzione che, dato un array a , restituisce un secondo array b con tutti e soli gli elementi dispari di a , nell'ordine in cui compaiono in a .
- Su stringhe
 - Funzione booleana che controlli se una stringa è palindroma ("osso").